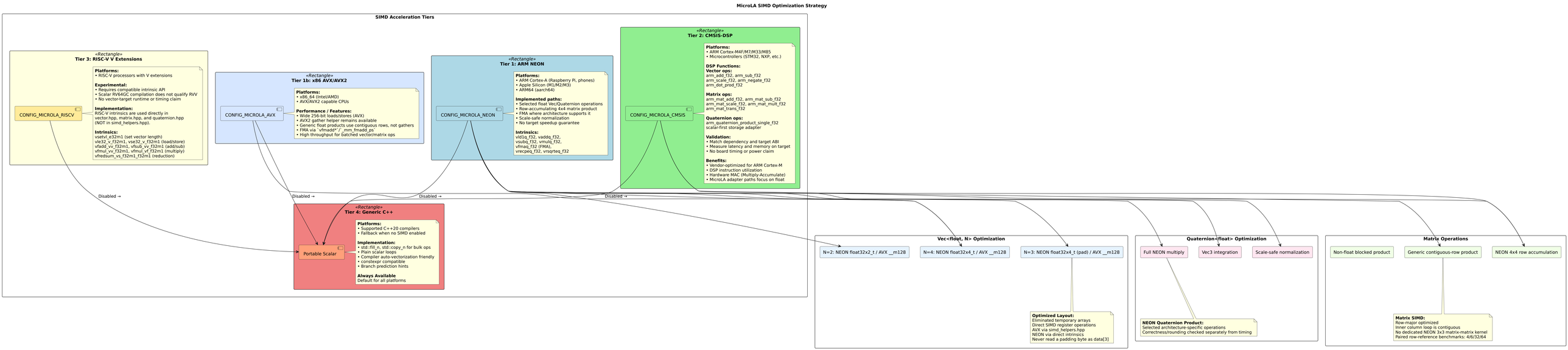
MicroLA SIMD Optimization Strategy
SIMD Acceleration Tiers
«Rectangle»
Tier 3: RISC-V V Extensions
Platforms:
• RISC-V processors with V extensions
Experimental:
• Requires compatible intrinsic API
• Scalar RV64GC compilation does not qualify RVV
• No vector-target runtime or timing claim
Implementation:
RISC-V intrinsics are used directly in vector.hpp, matrix.hpp, and quaternion.hpp (NOT in simd_helpers.hpp).
Intrinsics:
vsetvl_e32m1 (set vector length)
vle32_v_f32m1, vse32_v_f32m1 (load/store)
vfadd_vv_f32m1, vfsub_vv_f32m1 (add/sub)
vfmul_vv_f32m1, vfmul_vf_f32m1 (multiply)
vfredsum_vs_f32m1_f32m1 (reduction)
CONFIG_MICROLA_RISCV
«Rectangle»
Tier 1b: x86 AVX/AVX2
Platforms:
• x86_64 (Intel/AMD)
• AVX/AVX2 capable CPUs
Performance / Features:
• Wide 256-bit loads/stores (AVX)
• AVX2 gather helper remains available
• Generic float products use contiguous rows, not gathers
• FMA via `vfmadd* / _mm_fmadd_ps`
• High throughput for batched vector/matrix ops
CONFIG_MICROLA_AVX
«Rectangle»
Tier 1: ARM NEON
Platforms:
• ARM Cortex-A (Raspberry Pi, phones)
• Apple Silicon (M1/M2/M3)
• ARM64 (aarch64)
Implemented paths:
• Selected float Vec/Quaternion operations
• Row-accumulating 4x4 matrix product
• FMA where architecture supports it
• Scale-safe normalization
• No target speedup guarantee
Intrinsics:
vld1q_f32, vaddq_f32, vsubq_f32, vmulq_f32, vfmaq_f32 (FMA), vrecpeq_f32, vrsqrteq_f32
CONFIG_MICROLA_NEON
«Rectangle»
Tier 2: CMSIS-DSP
Platforms:
• ARM Cortex-M4f/M7/M33/M85
• Microcontrollers (STM32, NXP, etc.)
DSP Functions:
Vector ops:
arm_add_f32, arm_sub_f32
arm_scale_f32, arm_negate_f32
arm_dot_prod_f32
Matrix ops:
arm_mat_add_f32, arm_mat_sub_f32
arm_mat_scale_f32, arm_mat_mult_f32
arm_mat_trans_f32
Quaternion ops:
arm_quaternion_product_single_f32
scalar-first storage adapter
Validation:
• Match dependency and target ABI
• Measure latency and memory on target
• No board timing or power claim
Benefits:
• Vendor-optimized for ARM Cortex-M
• DSP instruction utilization
• Hardware MAC (Multiply-Accumulate)
• MicroLA adapter paths focus on float
CONFIG_MICROLA_CMSIS
«Rectangle»
Tier 4: Generic C++
Platforms:
• Supported C++20 compilers
• Fallback when no SIMD enabled
Implementation:
• std::fill_n, std::copy_n for bulk ops
• Plain scalar loops
• Compiler auto-vectorization friendly
• constexpr compatible
• Branch prediction hints
Always Available
Default for all platforms
Portable Scalar
Disabled →
Disabled →
Disabled →
Disabled →
Vec<float, N> Optimization
N=2: NEON float32x2_t / AVX __m128
N=4: NEON float32x4_t / AVX __m128
N=3: NEON float32x4_t (pad) / AVX __m128
Optimized Layout:
Eliminated temporary arrays
Direct SIMD register operations
AVX via simd_helpers.hpp
NEON via direct intrinsics
Never read a padding byte as data[3]
Quaternion<float> Optimization
Full NEON multiply
Vec3 integration
Scale-safe normalization
NEON Quaternion Product:
Selected architecture-specific operations
Correctness/rounding checked separately from timing
Matrix Operations
Non-float blocked product
Generic contiguous-row product
NEON 4x4 row accumulation
Matrix SIMD:
Row-major optimized
Inner column loop is contiguous
No dedicated NEON 3x3 matrix-matrix kernel
Paired row-reference benchmarks: 4/6/32/64